



FACULTAD DE INGENIERÍA

Carrera Profesional de Ingeniería de Sistemas e Informática

Carrera Profesional de Ingeniería de Software

CURSO: ALGORITMOS Y ESTRUCTURAS DE DATOS

Título Proyecto: Sistema de Búsquedas y Estadísticas de Investigadores CONCYTEC utilizando Tabla Hash Encadenada

Ciclo: 05 | 2026-MARZO

Integrante(s):

ID	CUI	APELLIDOS	NOMBRES
1	U24318514	Pineda Vilca	Hussein Emmanuel
2	U23241051	Prado Lozada	Alison Dayana

Docente: MBA. Mg. Ing. Rene Alonso Nieto Valencia.

Arequipa – Perú.

2026

ÍNDICE CONTENIDO

ÍNDICE CONTENIDO	2
ÍNDICE DE TABLAS.....	3
ÍNDICE GRÁFICOS E IMÁGENES.....	4
HISTORIAL DE VERSIONES.....	5
CAPÍTULO 1 ASPECTOS GENERALES	6
1.1. DEFINICIÓN DEL PROBLEMA.....	6
1.1.1. DESCRIPCIÓN DEL PROBLEMA	6
1.2. DEFINICIÓN DE OBJETIVOS.....	6
1.2.1. OBJETIVOS DEL SISTEMA	6
1.3. ALCANCE	7
1.3.1. DESCRIPCIÓN DE LOS FORMULARIOS	7
1.3.2. DESCRIPCIÓN DE LOS ARCHIVOS.....	12
CAPITULO 2 DISEÑO DE LA APLICACIÓN.....	14
2.1. DISEÑO GENERAL DEL SISTEMA.....	14
2.1.1. DISEÑO DE LAS CLASES.....	14
2.1.2. DISEÑO DE ESTRUCTURA DE DATOS UTILIZADA.....	15
2.1.3. OPERACIONES DE ESTRUCTURAS DE DATOS UTILIZADA.....	16
CONCLUSIONES.....	18
RECOMENDACIONES	18
BIBLIOGRAFÍA	19
ANEXOS.....	20

ÍNDICE DE TABLAS

Tabla 1. Formulario FrmPrincipal.....	8
Tabla 2. Eventos de FrmPrincipal.....	8
Tabla 3. Formulario FrmMantenimiento.....	9
Tabla 4. Eventos del FrmMantenimiento.....	9
Tabla 5. Formulario FrmBusqueda	10
Tabla 6. Eventos del FrmBusqueda.....	10
Tabla 7. Formulario FrmEstadísticas	11
Tabla 8. Eventos del FrmEstadísticas	11
Tabla 9. Descripción de archivos y paquetes Java	12
Tabla 10. Archivo csv para investigadores	12
Tabla 11. Archivo csv para centros autorizados.....	13

ÍNDICE GRÁFICOS E IMÁGENES

Ilustración 1. Diagrama de clases UML.....	15
Ilustración 2. Representación de t abla hash encadenada.....	16
Ilustración 3. Inserción de nodo en tabla hash	17
Ilustración 4. Búsqueda en una lista enlazada.....	17
Ilustración 5. Eliminación de nodo en lista enlazada	17

HISTORIAL DE VERSIONES

Fecha de Elaboración	Versión	Elaborado por	Descripción	Revisado por	Fecha de Revisión
13-07-2026	1.0	Equipo	Creación del documento incluyendo trabajo realizado sobre la aplicación		

CAPÍTULO 1 ASPECTOS GENERALES

1.1. DEFINICIÓN DEL PROBLEMA.

1.1.1. DESCRIPCIÓN DEL PROBLEMA

El Consejo Nacional de Ciencia, Tecnología e Innovación Tecnológica (CONCYTEC) administra información correspondiente a investigadores RENACYT y centros autorizados de investigación mediante archivos en formato CSV. Estos archivos contienen más de nueve mil registros, incluyendo datos como código RENACYT, nombre del investigador, género, condición de actividad, nivel, departamento, provincia, distrito y centro autorizado.

Actualmente, cuando la información permanece únicamente almacenada en archivos planos, las búsquedas, consultas y generación de reportes resultan lentas y poco eficientes. Además, mantener actualizados los registros implica modificar manualmente los archivos, aumentando la posibilidad de errores y pérdida de información.

Con el propósito de solucionar esta problemática se desarrolló una aplicación de escritorio en Java SE que permite importar los archivos CSV, almacenar los datos en memoria mediante una estructura de datos propia basada en una tabla hash encadenada, realizar operaciones CRUD, efectuar búsquedas rápidas con distintos filtros y generar estadísticas, todo ello sin utilizar bases de datos ni las colecciones estándar de Java.

1.2. DEFINICIÓN DE OBJETIVOS

1.2.1. OBJETIVOS DEL SISTEMA

- **Objetivo general:**

Desarrollar un sistema de escritorio para la gestión, búsqueda y análisis estadístico de investigadores RENACYT utilizando Java SE, Swing y una tabla hash encadenada propia, permitiendo consultar y administrar grandes volúmenes de información almacenada en archivos CSV.

- **Objetivos específicos:**

- Importar archivos CSV de investigadores y centros autorizados.
- Implementar una tabla hash encadenada genérica mediante listas enlazadas propias.
- Registrar nuevos investigadores.
- Actualizar información existente.
- Eliminar investigadores.
- Buscar investigadores por código RENACYT, nombre o fecha de constancia.
- Filtrar investigadores por centro autorizado, provincia y distrito.
- Consultar información de centros autorizados.
- Generar estadísticas por departamento, provincia, distrito, institución y áreas científicas.
- Exportar los resultados obtenidos a archivos CSV.

1.3. ALCANCE

El sistema desarrollado corresponde a una aplicación de escritorio implementada en Java SE utilizando la biblioteca Swing para la interfaz gráfica de usuario. Toda la información es almacenada en memoria mediante estructuras de datos desarrolladas desde cero, sin utilizar bases de datos ni colecciones del paquete java.util.

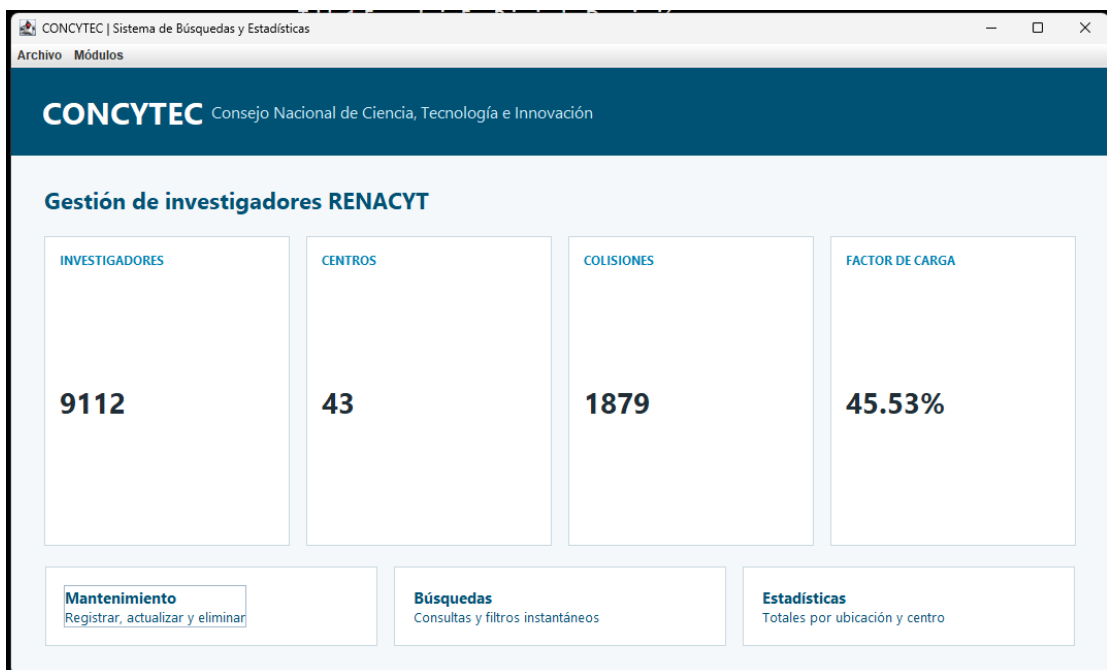
La solución implementa los siguientes módulos principales:

- Menú principal.
- Importación de archivos CSV.
- Mantenimiento de investigadores.
- Búsqueda de investigadores.
- Consulta de centros autorizados.
- Estadísticas.
- Exportación de resultados en CSV y PDF.

La estructura principal utilizada es una tabla hash encadenada cuya clave corresponde al código RENACYT de cada investigador, permitiendo búsquedas rápidas y un manejo eficiente de colisiones mediante listas enlazadas.

1.3.1. DESCRIPCIÓN DE LOS FORMULARIOS

Formulario 1. FrmPrincipal



Nombre del formulario	Descripción	Usuarios
FrmPrincipal	Permite acceder a todos los módulos principales del sistema. • Importar investigadores desde archivos CSV. • Importar centros autorizados. • Acceder al mantenimiento de investigadores. • Acceder al módulo de búsquedas. • Acceder al módulo de estadísticas. • Visualizar indicadores generales como cantidad de investigadores, centros autorizados, colisiones y factor de carga de la tabla hash.	• Administrador del sistema • Asistente de análisis

Tabla 1. Formulario FrmPrincipal

Descripción de los Eventos

Control	Título	Evento	Descripción
btnImportarInvestigadores	Registrar Cliente	Clic	Importa el archivo CSV de investigadores y carga los registros en la tabla hash.
btnImportarCentros	Modificar Cliente	Clic	Importa el archivo CSV de centros autorizados.
btnMantenimiento	Eliminar Cliente	Clic	Abre el formulario de mantenimiento de investigadores.
btnBusqueda	Búsquedas	Clic	Abre el módulo de consultas y filtros.
btnEstadisticas	Estadísticas	Clic	Abre el formulario de estadísticas del sistema.

Tabla 2. Eventos de FrmPrincipal

Formulario 2. FrmMantenimiento

The screenshot displays the CONCYTEC Sistema de Búsquedas y Estadísticas interface. A modal window titled 'Mantenimiento de investigadores' is open, showing a form for managing researchers. The form includes fields for 'Código RENACYT *', 'Investigador *', 'Género' (set to 'Femenino'), 'Condición' (set to 'Activo'), 'Fecha constancia', 'Nivel 2021' (set to 'I'), 'Departamento', 'Provincia', 'Distrito', and 'Centro autorizado'. At the bottom of the modal are buttons for 'Consultar', 'Registrar', 'Actualizar', 'Eliminar', and 'Limpiar'. The background interface shows a sidebar with 'Gestión de investigadores' and 'INVESTIGADORES' (9112), and a main area with 'CTOR DE CARGA' (5.53%).

Nombre del formulario	Descripción	Usuarios
FrmMantenimiento	Permite administrar los investigadores registrados en el sistema mediante operaciones CRUD. • Consultar investigador. • Registrar nuevos investigadores. • Actualizar información existente. • Eliminar investigadores. • Limpiar los controles del formulario.	• Administrador del sistema.

Tabla 3. Formulario FrmMantenimiento

Descripción de los Eventos

Control	Título	Evento	Descripción
btnConsultar	Consultar investigador	Clic	Busca un investigador mediante su código RENACYT.
btnRegistrar	Registrar investigador	Clic	Inserta un nuevo investigador en la tabla hash y actualiza el archivo CSV.
btnActualizar	Actualizar investigador	Clic	Modifica la información del investigador seleccionado.
btnEliminar	Eliminar investigador	Clic	Elimina el investigador de la tabla hash y del archivo CSV.
btnLimpiar	Limpiar formulario	Clic	Borra todos los datos ingresados en el formulario.

Tabla 4. Eventos del FrmMantenimiento

Formulario 3. FrmBusqueda

Código RENACYT	Investigador	Género	Condición	Nivel	Departamento	Provincia	Distrito	Centro
P0083759	VILLEGAS AGUIL...	Femenino	Activo	VII	PIURA	PIURA	PIURA	10
P0108921	VASQUEZ VELAS...	Femenino	Activo	IV	LIMA	LIMA	SANTIA DE SUR...	40
P0023559	REVOLLEDO PIZ...	Femenino	VIGENCIA VENCI...					28
P0064388	QUEVEDO SANC...	Femenino	Activo	VI	LA LIBERTAD	TRUJILLO	TRUJILLO	35
P0004185	MATTA SOLIS,HE...	Masculino	Activo	II	LIMA	LIMA	LOS OLIVOS	24
P0013899	VELAZCO CASTR...	Femenino	Activo	VII	UCAYALI	CORONEL PORT...	CALLERIA	34
P0004180	TRUJILLO QUIN...	Masculino	Activo	VII	LIMA	LIMA	LIMA	21
P0049091	ORMENO AYALA...	Femenino	Activo	VII	CUSCO	CUSCO	SANTIA	41
P0017977	ACHANCARAY ...	Masculino	Activo	VI				19
P0155326	CHURA VILCAN...	Masculino	Activo	IV	PUNO	PUNO	PUNO	27
P0013893	CRUZ DE LA CR...	Masculino	Activo	VI	PUNO	PUNO	PUNO	33
P0013892	BIAMONT ROJA...	Masculino	Activo	VI	PUNO	PUNO	PUNO	31
P0155322	OCHATOMA PA...	Masculino	Activo	VI	PUNO	PUNO	PUNO	26
P0017970	IBANEZ ERQUIA...	Masculino	Activo	VI	LIMA	LIMA	SAN ISIDRO	18
P0100752	TELLEZ CHIRINO...	Masculino	Activo	V	LIMA	LIMA	LA MOLINA	13
P0000099	MAMANI MACE...	Masculino	VIGENCIA VENCI...		PROV. CONST. D...	CALLAO	CARMEN DE LA ...	25
P0000098	ZIMIC PERALTA...	Masculino	Activo	Investigador Dis...	LIMA	LIMA	SAN BORJA	14
P0000097	MARCELO ALDA...	Masculino	Activo	III	PIURA	PIURA	PIURA	40
P0000096	SERRANO MART...	Masculino	Activo	III	LIMA	LIMA	JESUS MARIA	17
P0000095	GOMEZ PANDO...	Femenino			LIMA	LIMA	LA MOLINA	16
P0004178	TINOCO GOMEZ...	Masculino	Excluido		LIMA	LIMA	LOS OLIVOS	11
P0083745	VILCA HORNA,N...	Femenino	Activo	VI	LIMA	LIMA	SAN JUAN DE L...	9
P0064376	RAFAEL HEREDIA...	Masculino	Activo	VI	UCAYALI	CORONEL PORT...	YARINACOCOA	34
P0213153	CAMPOS QUIRO...	Femenino	Activo	VII	MOQUEGUA	ILO	PACOCOA	29
P0064375	SANTOS ROSAL...	Femenino	Activo	VII	LIMA	LIMA	SAN JUAN DE L...	33
P0042914	CRUZALEGUI FE...	Masculino	Activo	VII	AMAZONAS	UTCUBAMBA	BAGUA GRANDE	29

Nombre del formulario	Descripción	Usuarios
FrmBusqueda	Permite realizar consultas de investigadores y centros autorizados utilizando distintos criterios. • Buscar por código RENACYT. • Buscar por investigador. • Buscar por fecha de constancia. • Filtrar por centro autorizado, provincia y distrito. • Exportar resultados en CSV y PDF	• Administrador del sistema • Analista

Tabla 5. Formulario FrmBusqueda

Descripción de los Eventos

Control	Título	Evento	Descripción
txtBusqueda	Buscar	Cambio de texto	Actualiza automáticamente los resultados de búsqueda.
cbmCampo	Seleccionar criterio	Cambio	Cambia el criterio utilizado para la búsqueda.
chkCentro	Filtrar por centro	Clic	Activa o desactiva el filtro por centro autorizado.
chkProvincia	Filtrar por provincia	Clic	Activa o desactiva el filtro por provincia.
chkDistrito	Filtrar por distrito	Clic	Activa o desactiva el filtro por distrito.
btnExportar	Exportar resultados	Clic	Exporta la información mostrada a archivos CSV y PDF.

Tabla 6. Eventos del FrmBusqueda

Formulario 4. FrmEstadisticas

Categoría	Total de investigadores
TUMBES	44
MADRE DE DIOS	53
UCAYALI	73
HUANCAVELICA	68
TACNA	145
PASCO	27
LORETO	149
APURIMAC	105
AYACUCHO	72
SAN MARTIN	86
AREQUIPA	553
Sin especificar	1
LAMBAYEQUE	294
PROV. CONST. DEL CALLAO	152
ICA	106
AMAZONAS	135
PUNO	337
CAJAMARCA	108
MOQUEGUA	48
CUSCO	180
JUNIN	290
HUANUCO	202
PIURA	216
LA LIBERTAD	561
ANCASH	152
LIMA	4559

Nombre del formulario	Descripción	Usuarios
FrmEstadisticas	Permite generar reportes estadísticos a partir de la información almacenada en el sistema. • Estadísticas por departamento. • Provincia. • Distrito. • Institución. • Área científica. • Subárea científica. • Disciplina científica. • Exportar estadísticas en CSV y PDF.	• Administrador del sistema • Analista

Tabla 7. Formulario FrmEstadisticas

Descripción de los Eventos

Control	Título	Evento	Descripción
cmbTipo	Seleccionar categoría	Cambio	Permite elegir la categoría para generar la estadística.
btnCalcular	Calcular estadística	Clic	Calcula la estadística correspondiente a la categoría seleccionada.
btnExportar	Exportar estadística	Clic	Exporta los resultados obtenidos a archivos CSV y PDF.

Tabla 8. Eventos del FrmEstadisticas

1.3.2. DESCRIPCIÓN DE LOS ARCHIVOS

El sistema está organizado en archivos fuente Java distribuidos en paquetes según su responsabilidad y utiliza archivos CSV para almacenar la información persistente de investigadores y centros autorizados. Los archivos Java implementan la lógica del sistema, mientras que los archivos CSV permiten conservar la información aun después de cerrar la aplicación.

Archivos Java

Archivo / Paquete	Descripción
Main.java	Punto de entrada de la aplicación. Inicializa el sistema y carga la interfaz principal.
controlador	Contiene la lógica del sistema, incluyendo la importación, exportación, mantenimiento de datos y comunicación entre la interfaz y las estructuras de datos.
modelo	Define las entidades del sistema como Investigador, CentroAutorizado y EstadisticaRegistro.
tads	Implementa las estructuras de datos desarrolladas desde cero: Nodo, ListaEnlazada, TablaHashEncadenada y Visitante.
vista	Contiene todos los formularios Swing, modelos de tablas y configuración visual de la aplicación.

Tabla 9. Descripción de archivos y paquetes Java

Archivos de datos

El sistema utiliza dos archivos CSV como almacenamiento permanente de la información.

Archivo	Descripción
investigadores_concytec.csv	Contiene toda la información de los investigadores RENACYT. Se importa al iniciar el sistema y se actualiza automáticamente cada vez que se registra, modifica o elimina un investigador.

Tabla 10. Archivo csv para investigadores

Estructura del archivo. Campos:

- codigo_renacyt
- investigador
- genero
- URL_ficha_renacyt
- Reglamento
- condicion_actividad
- emision_de_constancia_reglamento_2021

- nivel_reglamento_2021
- grupo_reglamento_2018
- nivel_reglamento_2018
- fecha_inicio_vigencia_reglamento_2018
- fecha_fin_vigencia_reglamento_2018
- rango_edad
- pais_residencia
- departamento
- provincia
- distrito
- centro_autorizado

Archivo	Descripción
centros_autorizados_2023.csv	Almacena la información de los centros autorizados registrados por CONCYTEC. Su contenido es utilizado para las consultas y la generación de estadísticas relacionadas con instituciones y áreas científicas.

Tabla 11. Archivo csv para centros autorizados

Estructura del archivo. Campos:

- codigo
- resolucion
- nombre_institucion
- area_cientifica
- subarea_cientifica
- disciplina_cientifica
- investigacion_cientifica
- desarrollo_tecnologico
- innovacion_tecnologica

Persistencia de la información

La persistencia de los datos es administrada por la clase GestorArchivos, la cual se encarga de importar la información desde los archivos CSV, cargarla en la tabla hash implementada por el sistema y guardar nuevamente los registros cuando se realizan operaciones de registro, actualización o eliminación. De esta manera, los cambios efectuados por el usuario permanecen almacenados para futuras ejecuciones de la aplicación.

CAPITULO 2 DISEÑO DE LA APLICACIÓN

2.1. DISEÑO GENERAL DEL SISTEMA

La aplicación fue desarrollada siguiendo una arquitectura modular, separando las responsabilidades en paquetes independientes para facilitar el mantenimiento y la escalabilidad del software. Cada paquete agrupa clases con funciones específicas, permitiendo que la interfaz gráfica, la lógica del negocio, las entidades del sistema y las estructuras de datos trabajen de forma desacoplada.

La interacción comienza en la interfaz gráfica desarrollada con Java Swing, donde el usuario ejecuta las diferentes operaciones del sistema. Estas solicitudes son procesadas por la clase ServicioConcytec, la cual coordina el acceso a los datos almacenados en una tabla hash encadenada y administra la lectura y escritura de los archivos CSV mediante la clase GestorArchivos. Gracias a esta organización, el sistema mantiene una clara separación entre la presentación, la lógica de negocio, la persistencia y las estructuras de datos implementadas.

2.1.1. DISEÑO DE LAS CLASES

El sistema está organizado en cuatro paquetes principales: vista, controlador, modelo y tads, además de la clase principal encargada de iniciar la aplicación. Esta organización permite distribuir adecuadamente las responsabilidades y favorecer la reutilización del código.

El paquete “*vista*” contiene todos los formularios desarrollados con Java Swing, entre ellos FrmPrincipal, FrmMantenimiento, FrmBusqueda y FrmEstadísticas, además de las clases auxiliares utilizadas para mostrar la información en las tablas y configurar el aspecto visual de la aplicación.

El paquete “*controlador*” concentra la lógica del sistema mediante las clases ServicioConcytec, GestorArchivos y ArchivoControlador. Estas clases son responsables de importar y exportar archivos CSV, realizar las operaciones CRUD sobre los investigadores, ejecutar las búsquedas y generar las estadísticas solicitadas por el usuario.

El paquete “*modelo*” representa las entidades principales del sistema mediante las clases Investigador, CentroAutorizado y EstadisticaRegistro, las cuales encapsulan los atributos correspondientes a cada tipo de información.

Finalmente, el paquete “*tads*” implementa las estructuras de datos desarrolladas desde cero para el proyecto, incluyendo las clases Nodo, ListaEnlazada, TablaHashEncadenada y la interfaz Visitante, permitiendo almacenar y recorrer la información sin utilizar las colecciones estándar de Java.

Las listas enlazadas están compuestas por nodos que almacenan la clave, el objeto asociado y una referencia al siguiente nodo. Gracias a esta implementación fue posible cumplir con el requisito del proyecto de desarrollar estructuras de datos propias, evitando el uso de colecciones proporcionadas por la biblioteca estándar de Java.

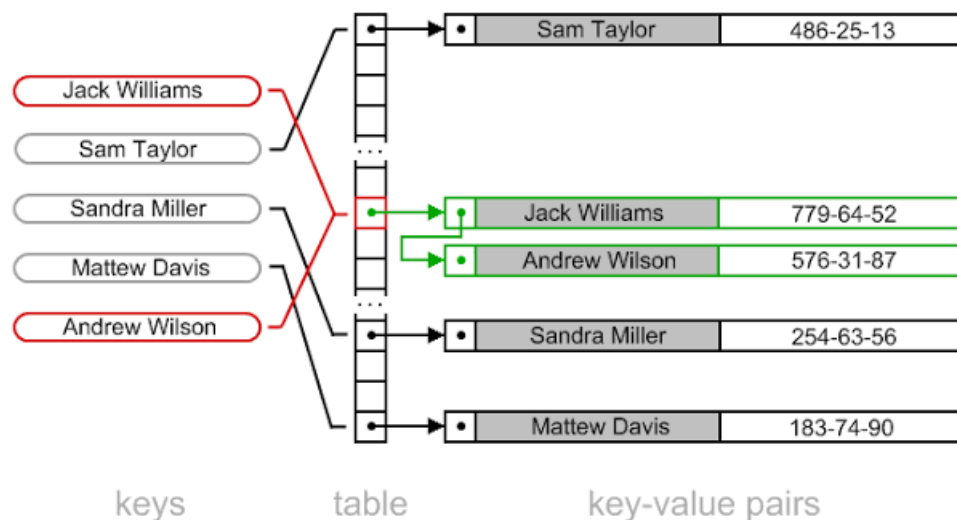


Ilustración 2. Representación de tabla hash encadenada

2.1.3. OPERACIONES DE ESTRUCTURAS DE DATOS UTILIZADA

Las operaciones principales del sistema se realizan directamente sobre la tabla hash encadenada. Para registrar un investigador se calcula el índice correspondiente mediante la función hash y el nuevo nodo se inserta dentro de la lista enlazada asociada a esa cubeta. Si la clave ya existe, el registro es reemplazado por la nueva información.

La búsqueda por código RENACYT utiliza el mismo procedimiento, calculando primero el índice hash y recorriendo únicamente la lista enlazada correspondiente hasta encontrar el investigador solicitado. Esta estrategia proporciona un tiempo promedio de búsqueda de $O(1)$.

La actualización consiste en localizar el investigador mediante su clave y reemplazar la información almacenada, mientras que la eliminación recorre la lista enlazada de la cubeta correspondiente hasta encontrar el nodo que contiene la clave buscada, reajustando posteriormente las referencias entre nodos para mantener la integridad de la estructura.

En aquellas consultas donde el criterio de búsqueda no corresponde a la clave principal —como nombre del investigador, fecha de constancia, provincia o distrito— el sistema recorre todas las cubetas de la tabla hash evaluando cada registro almacenado hasta construir la lista de resultados. Posteriormente, la información puede exportarse a archivos CSV o PDF mediante las funcionalidades implementadas en el sistema.

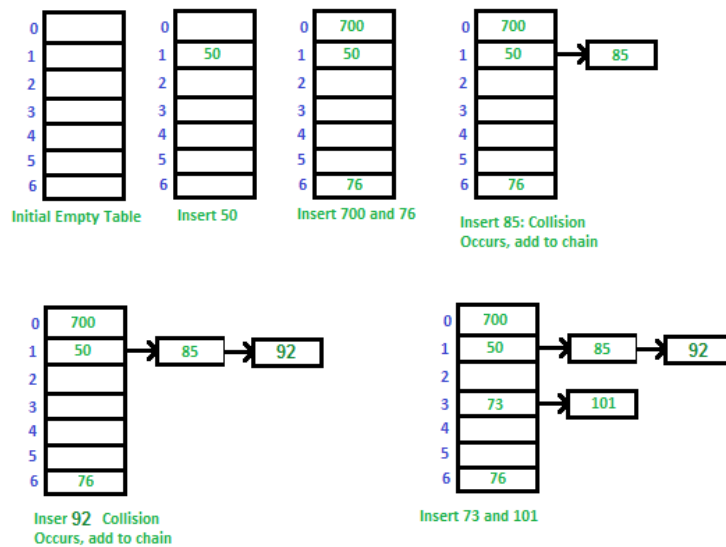


Ilustración 3. Inserción de nodo en tabla hash

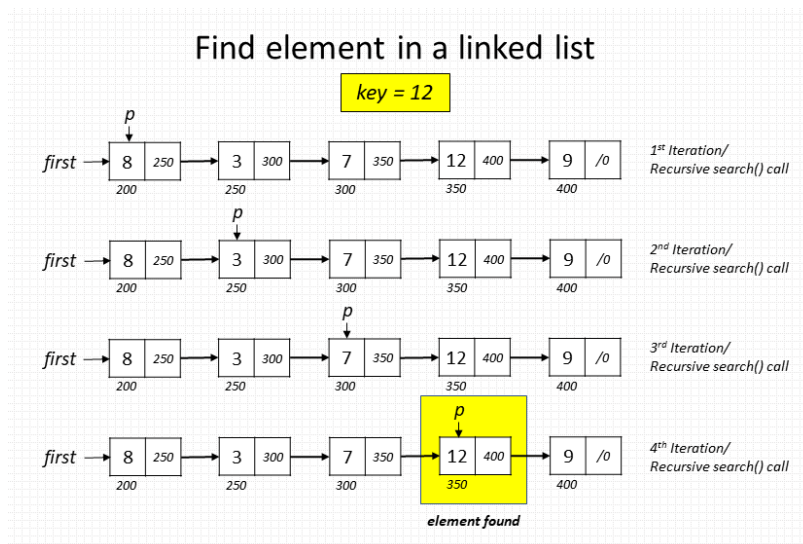


Ilustración 4. Búsqueda en una lista enlazada

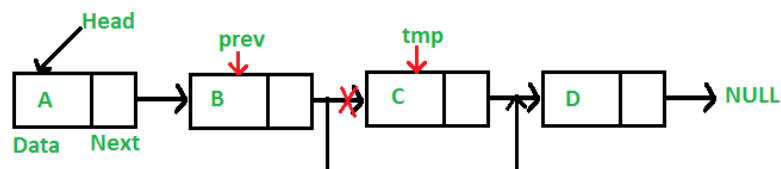


Ilustración 5. Eliminación de nodo en lista enlazada

CONCLUSIONES

Primera: Se desarrolló satisfactoriamente un sistema de escritorio en Java SE que permite gestionar investigadores RENACYT mediante una tabla hash encadenada implementada desde cero, cumpliendo con los requisitos establecidos para el proyecto del curso.

Segunda: La implementación de una tabla hash utilizando listas enlazadas permitió resolver eficientemente las colisiones, obteniendo búsquedas y actualizaciones con un tiempo promedio cercano a $O(1)$ para las consultas por código RENACYT.

Tercera: El uso de archivos CSV como mecanismo de persistencia permitió almacenar la información de investigadores y centros autorizados sin necesidad de utilizar un sistema gestor de bases de datos, manteniendo la simplicidad y portabilidad de la aplicación.

Cuarta: La separación del proyecto en paquetes de presentación, controlador, modelo y estructuras de datos facilitó la organización del código, promoviendo una arquitectura modular y de fácil mantenimiento.

Quinta: Las funcionalidades de mantenimiento, búsquedas, filtros, generación de estadísticas y exportación de resultados demuestran la aplicación práctica de las estructuras de datos estudiadas durante el curso, integrándolas en una solución funcional para el análisis de información científica.

RECOMENDACIONES

Primera: Validar previamente la estructura y el formato de los archivos CSV antes de importarlos, con el fin de evitar errores ocasionados por registros incompletos o inconsistentes.

Segunda: Mantener copias de seguridad de los archivos de datos antes de realizar operaciones de actualización o eliminación, garantizando la recuperación de la información ante posibles errores.

Tercera: Como trabajo futuro, incorporar mecanismos de ordenamiento de resultados y nuevas opciones de búsqueda que permitan consultas aún más flexibles.

Cuarta: Implementar una interfaz de autenticación de usuarios para restringir el acceso a las operaciones de mantenimiento y proteger la integridad de la información almacenada.

Quinta: Ampliar el sistema para soportar otros formatos de intercambio de información, como archivos Excel o bases de datos relacionales, conservando la tabla hash como estructura principal para el procesamiento en memoria.

BIBLIOGRAFÍA

- [1] T. H. Cormen, C. E. Leiserson, R. L. Rivest y C. Stein, Introduction to Algorithms, 4th ed. Cambridge, MA, USA: MIT Press, 2022.
- [2] R. Sedgewick y K. Wayne, Algorithms, 4th ed. Boston, MA, USA: Addison-Wesley Professional, 2011.
- [3] M. T. Goodrich, R. Tamassia y M. H. Goldwasser, Data Structures and Algorithms in Java, 6th ed. Hoboken, NJ, USA: John Wiley & Sons, 2014.
- [4] Oracle Corporation, Java Platform, Standard Edition 17 API Specification. [En línea]. Disponible: <https://docs.oracle.com/en/java/javase/17/docs/api/>
- [5] Consejo Nacional de Ciencia, Tecnología e Innovación Tecnológica (CONCYTEC), "Registro Nacional Científico, Tecnológico y de Innovación Tecnológica (RENACYT)", datos abiertos utilizados para el desarrollo del proyecto.

ANEXOS

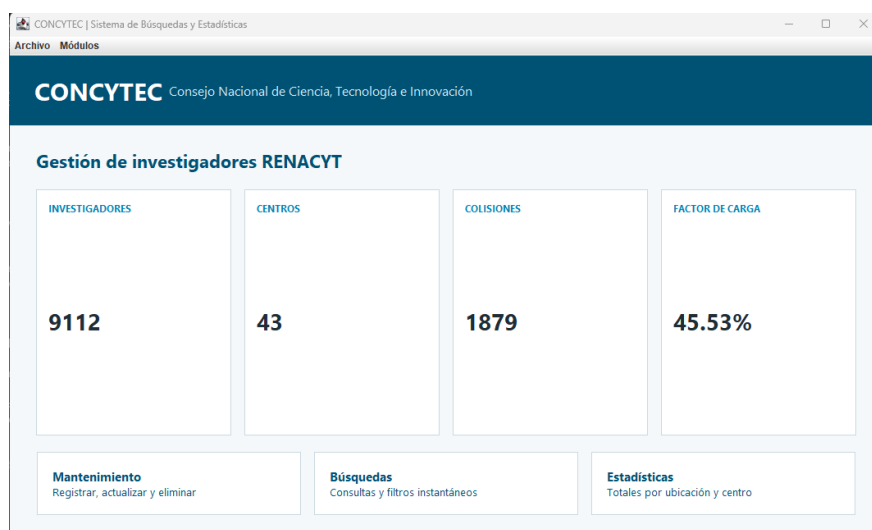
Anexo A. Estructura del proyecto

```
src/
├── Main.java
├── controlador/
│   ├── ArchivoControlador.java
│   ├── GestorArchivos.java
│   └── ServicioConcytec.java
├── modelo/
│   ├── Investigador.java
│   ├── CentroAutorizado.java
│   └── EstadisticaRegistro.java
├── tads/
│   ├── Nodo.java
│   ├── ListaEnlazada.java
│   ├── TablaHashEncadenada.java
│   └── Visitante.java
├── vista/
│   ├── FrmPrincipal.java
│   ├── FrmMantenimiento.java
│   ├── FrmBusqueda.java
│   ├── FrmEstadisticas.java
│   ├── ModeloTablaInvestigadores.java
│   ├── ModeloTablaEstadisticas.java
│   └── TemaConcytec.java
```

Anexo B. Archivos utilizados para los datos

- **investigadores_concytec.csv:** contiene los registros de investigadores RENACYT.
- **centros_autorizados_2023.csv:** contiene la información de los centros autorizados utilizados para búsquedas y estadísticas.

Anexo C. Capturas del sistema



CONCYTEC Consejo Nacional de Ciencia, Tecnología e Innovación

Gestión de investigadores RENACYT

INVESTIGADORES	CENTROS	COLISIONES	FACTOR DE CARGA
9112	43	1879	45.53%

Mantenimiento
Registrar, actualizar y eliminar

Búsquedas
Consultas y filtros instantáneos

Estadísticas
Totales por ubicación y centro

Mantenimiento de investigadores

Código RENACYT *
Investigador *
Género: Femenino
Condición: Activo
Fecha constancia:
Nivel 2021: I
Departamento:
Provincia:
Distrito:
Centro autorizado:

Consultar Registrar Actualizar Eliminar Limpiar

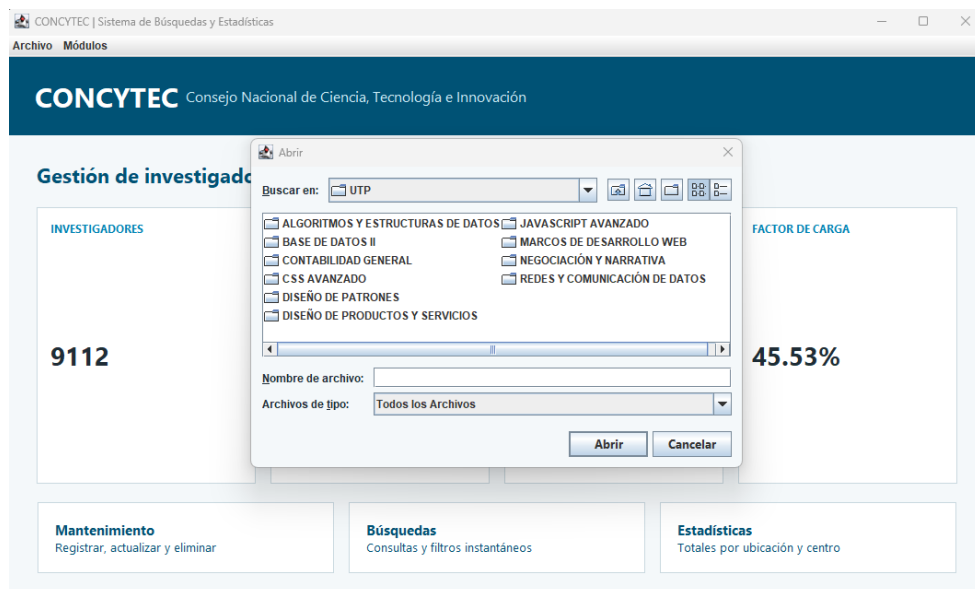
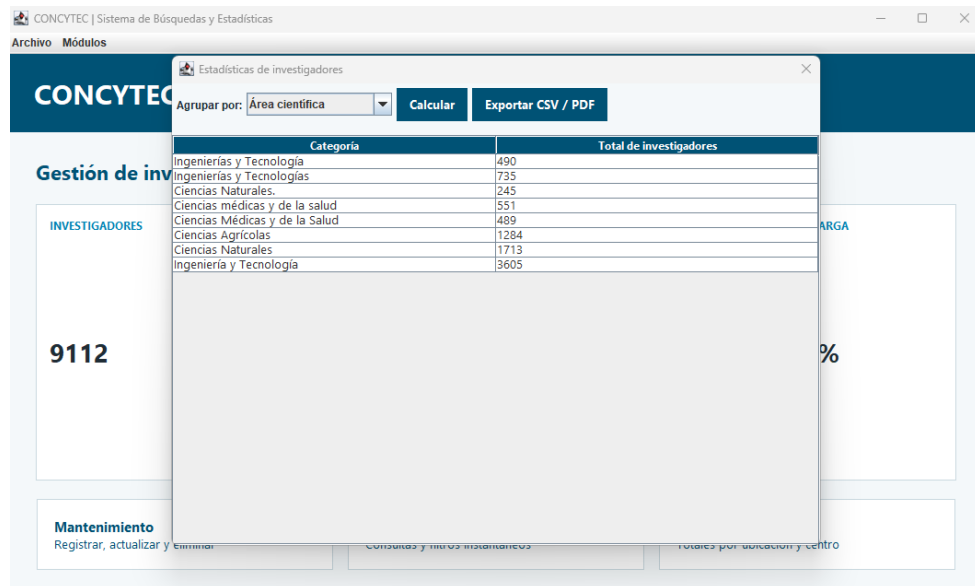
Búsquedas CONCYTEC

A Investigadores Centros autorizados

Buscar por: Código RENACYT Texto: P0083759 Exportar CSV / PDF

☐ Centro ☐ Provincia ☐ Distrito

Código RENACYT	Investigador	Género	Condición	Nivel	Departamento	Provincia	Distrito	Centro
P0083759	VILLEGAS AGUILAR, F...	Femenino	Activo	VII	PIURA	PIURA	PIURA	10



ANEXO D. Fragmentos de código fuente

Repositorio del proyecto disponible en Github:

<https://github.com/HusseinPineda02/PF-ALGORITMOS/tree/main>

Clase TablaHashEncadenada

```
public final class TablaHashEncadenada<K, V> {
    private final ListaEnlazada<K, V>[] cubetas;
    private int cantidad;
    private int colisiones;

    @SuppressWarnings("unchecked")
    public TablaHashEncadenada(int capacidad) {
        cubetas = (ListaEnlazada<K, V>[]) new
        ListaEnlazada[capacidad];
    }
}
```

```

        int i = 0;
        while (i < capacidad) { cubetas[i] = new ListaEnlazada<K,
V>(); i++; }
    }

    public void poner(K clave, V valor) {
        validarClave(clave);
        ListaEnlazada<K, V> lista = cubetas[indice(clave)];
        boolean habiaElementos = lista.tamano() > 0;
        if (lista.insertarOReemplazar(clave, valor)) {
            cantidad++;
            if (habiaElementos) colisiones++;
        }
    }

    public V obtener(K clave) { return clave == null ? null :
cubetas[indice(clave)].obtener(clave); }
    public V eliminar(K clave) {
        if (clave == null) return null;
        ListaEnlazada<K, V> lista = cubetas[indice(clave)];
        V eliminado = lista.eliminar(clave);
        if (eliminado != null) { cantidad--; if (lista.tamano() >
0 && colisiones > 0) colisiones--; }
        return eliminado;
    }

    public boolean contiene(K clave) { return obtener(clave) !=
null; }
    public int tamano() { return cantidad; }
    public int capacidad() { return cubetas.length; }
    public int colisiones() { return colisiones; }
    public double factorCarga() { return (double) cantidad /
cubetas.length; }
    @SuppressWarnings("unchecked")
    public void limpiar() {
        int i=0; while(i<cubetas.length){ cubetas[i]=new
ListaEnlazada<K,V>(); i++; }
        cantidad=0; colisiones=0;
    }

    public void recorrer(Visitante<K, V> visitante) {
        int i = 0; while (i < cubetas.length) {
cubetas[i].recorrer(visitante); i++; }
    }

    private int indice(K clave) {
        String texto = String.valueOf(clave);
        int hash = 0;
        int i = 0;
        while (i < texto.length()) { hash = 31 * hash +
texto.charAt(i); i++; }
        return (hash & 0x7fffffff) % cubetas.length;
    }

    private void validarClave(K clave) { if (clave == null) throw
new IllegalArgumentException("La clave no puede ser nula."); }
}

```

ANEXO E. Diagramas e ilustraciones empleados

